

Preliminary Statistical Testing of SPFL Dataset

2026-08-20

Contents

Introduction	1
Load packages and datasets	1
Data cleaning/prep	2
Visualisation of match outcomes	8
Visualisation of home and away goals	10
Attendance and match outcome	14
Visualisation of attendance by match result	14
Logistic regression: do percent capacity and travel distance affect the probability of a home win?	15
Visualisation of percent capacity and home wins	16
Visualisation of travel distance and home wins	17
Disciplinary analysis: did away teams receive more cards?	18
Distribution of yellow cards per match	18
Visualisation of yellow cards	20
Red cards	22
Visualisation of red cards	23
Key Match incident(KMI) data for 2024-25, 2025-26	24

Introduction

The aim is to conduct a preliminary statistical investigation into home advantage in the SPFL. The analysis explores whether home teams win or score more often, whether attendance and away-team travel distance are associated with home wins, and whether disciplinary patterns differ between home and away teams. These tests are exploratory and are used to identify patterns rather than prove causation.

Load packages and datasets

```
# Load the packages required for importing Excel files, data manipulation,  
# and producing plots.  
library(readxl)  
library(dplyr)  
library(ggplot2)  
  
# Define a function to load one season of SPFL data.  
# The function reads an Excel file, optionally from a specified sheet,  
# and adds a Season column so that each dataset can be identified later.  
load_season <- function(file, season, sheet = NULL) {
```

```

# If no sheet name is supplied, read the first sheet in the Excel file.
if (is.null(sheet)) {
  data <- read_excel(file)
} else {
  # If a sheet name is supplied, read that specific sheet.
  data <- read_excel(file, sheet = sheet)
}

# Add a Season column to the dataset.
data %>%
  mutate(Season = season)
}

# Load each individual SPFL Premiership season dataset.
# The sheet argument is included where the relevant Excel file contains
# a named worksheet that needs to be specified.
spfl_2016_2017 <- load_season("SPFL 2016-2017.xlsx", "2016/17", "SPFL 2016-2017")
spfl_2017_2018 <- load_season("SPFL 2017-2018.xlsx", "2017/18", "SPFL 2017-2018")
spfl_2018_2019 <- load_season("SPFL 2018-2019.xlsx", "2018/19", "SPFL 2018-2019")
spfl_2019_2020 <- load_season("SPFL 2019-2020.xlsx", "2019/20", "SPFL 2019-2020")
spfl_2020_2021 <- load_season("SPFL 2020-2021.xlsx", "2020/21", "SPFL 2020-2021")
spfl_2021_2022 <- load_season("SPFL 2021-2022.xlsx", "2021/22")
spfl_2022_2023 <- load_season("SPFL 2022-2023.xlsx", "2022/23")
spfl_2023_2024 <- load_season("SPFL 2023-2024.xlsx", "2023/24")
spfl_2024_2025 <- load_season("SPFL 2024-2025.xlsx", "2024/25")
spfl_2025_2026 <- load_season("SPFL 2025-2026.xlsx", "2025/26", "SPFL Dataset")

# Store all ten season datasets in a named list.
# This makes it easier to apply the same analysis to each season separately.
season_list <- list(
  "2016/17" = spfl_2016_2017,
  "2017/18" = spfl_2017_2018,
  "2018/19" = spfl_2018_2019,
  "2019/20" = spfl_2019_2020,
  "2020/21" = spfl_2020_2021,
  "2021/22" = spfl_2021_2022,
  "2022/23" = spfl_2022_2023,
  "2023/24" = spfl_2023_2024,
  "2024/25" = spfl_2024_2025,
  "2025/26" = spfl_2025_2026
)

```

Data cleaning/prep

```

column_names_table <- bind_rows(
  lapply(names(season_list), function(season_name) {
    data.frame(
      Season = season_name,
      ColumnName = names(season_list[[season_name]])
    )
  })
)

```

column_names_table

##	Season	ColumnName
## 1	2016/17	Date
## 2	2016/17	Home Team
## 3	2016/17	Away Team
## 4	2016/17	Home Goals
## 5	2016/17	Away Goals
## 6	2016/17	Result
## 7	2016/17	Season
## 8	2017/18	Date
## 9	2017/18	Home Team
## 10	2017/18	Away Team
## 11	2017/18	Home Goals
## 12	2017/18	Away Goals
## 13	2017/18	Result
## 14	2017/18	Season
## 15	2018/19	Date
## 16	2018/19	Home Team
## 17	2018/19	Away Team
## 18	2018/19	Home Goals
## 19	2018/19	Away Goals
## 20	2018/19	Result
## 21	2018/19	Season
## 22	2019/20	Date
## 23	2019/20	Home Team
## 24	2019/20	Away Team
## 25	2019/20	Home Goals
## 26	2019/20	Away Goals
## 27	2019/20	Result
## 28	2019/20	Season
## 29	2020/21	Date
## 30	2020/21	Home Team
## 31	2020/21	Away Team
## 32	2020/21	Home Goals
## 33	2020/21	Away Goals
## 34	2020/21	Result
## 35	2020/21	Goal Difference
## 36	2020/21	Home Yellow cards
## 37	2020/21	Home Red cards
## 38	2020/21	Away Yellow cards
## 39	2020/21	Away Red cards
## 40	2020/21	Season
## 41	2021/22	Date
## 42	2021/22	Home Team
## 43	2021/22	Away Team
## 44	2021/22	Home Goals
## 45	2021/22	Away Goals
## 46	2021/22	Result
## 47	2021/22	Season
## 48	2022/23	Date
## 49	2022/23	Home Team
## 50	2022/23	Away Team
## 51	2022/23	Home Goals

```

## 52 2022/23          Away Goals
## 53 2022/23          Result
## 54 2022/23          Season
## 55 2023/24          Date
## 56 2023/24          Home Team
## 57 2023/24          Away Team
## 58 2023/24          Home Goals
## 59 2023/24          Away Goals
## 60 2023/24          Result
## 61 2023/24          Season
## 62 2024/25          Date
## 63 2024/25          Home Team
## 64 2024/25          Away Team
## 65 2024/25          Home Goals
## 66 2024/25          Away Goals
## 67 2024/25          Result
## 68 2024/25          Overall Attendance
## 69 2024/25          Teams
## 70 2024/25          Incorrect Decision for
## 71 2024/25          Incorrect Decision against
## 72 2024/25          Season
## 73 2025/26          Date
## 74 2025/26          Home Team
## 75 2025/26          Away Team
## 76 2025/26          Home Goals
## 77 2025/26          Away Goals
## 78 2025/26          Result
## 79 2025/26          Attendance
## 80 2025/26          Percent Capacity
## 81 2025/26          Travel Distance for Away Team
## 82 2025/26          Travel time for Away Team
## 83 2025/26          Teams
## 84 2025/26          Incorrect Decision for
## 85 2025/26          Incorrect Decision against
## 86 2025/26          SPFL team
## 87 2025/26          Stadium Capacity
## 88 2025/26          Team A
## 89 2025/26          Team B
## 90 2025/26          Distance via time Travelled (hrs)
## 91 2025/26          Distance between A and B (miles)
## 92 2025/26          Goal Difference
## 93 2025/26          Home Red cards
## 94 2025/26          Home Yellow cards
## 95 2025/26          Away Red cards
## 96 2025/26          Away Yellow cards
## 97 2025/26          Season

```

```

unique_column_names <- sort(unique(unlist(
  lapply(season_list, names)
)))

```

```
unique_column_names
```

```

## [1] "Attendance"          "Away Goals"
## [3] "Away Red cards"      "Away Team"

```

```
## [5] "Away Yellow cards" "Date"
## [7] "Distance between A and B (miles)" "Distance via time Travelled (hrs)"
## [9] "Goal Difference" "Home Goals"
## [11] "Home Red cards" "Home Team"
## [13] "Home Yellow cards" "Incorrect Decision against"
## [15] "Incorrect Decision for" "Overall Attendance"
## [17] "Percent Capacity" "Result"
## [19] "Season" "SPFL team"
## [21] "Stadium Capacity" "Team A"
## [23] "Team B" "Teams"
## [25] "Travel Distance for Away Team" "Travel time for Away Team"
```

```
team_names_table <- bind_rows(
  lapply(names(season_list), function(season_name) {

    teams <- sort(unique(c(
      season_list[[season_name]]$`Home Team`,
      season_list[[season_name]]$`Away Team`
    )))

    data.frame(
      Season = season_name,
      Team = teams
    )
  })
)
```

```
team_names_table
```

```
##      Season      Team
## 1  2016/17  Aberdeen
## 2  2016/17   Celtic
## 3  2016/17   Dundee
## 4  2016/17  Hamilton
## 5  2016/17   Hearts
## 6  2016/17 Inverness CT
## 7  2016/17  Kilmarnock
## 8  2016/17  Motherwell
## 9  2016/17 Partick Thistle
## 10 2016/17   Rangers
## 11 2016/17  Ross County
## 12 2016/17 St Johnstone
## 13 2017/18  Aberdeen
## 14 2017/18   Celtic
## 15 2017/18   Dundee
## 16 2017/18  Hamilton
## 17 2017/18   Hearts
## 18 2017/18  Hibernian
## 19 2017/18  Kilmarnock
## 20 2017/18  Motherwell
## 21 2017/18 Partick Thistle
## 22 2017/18   Rangers
## 23 2017/18  Ross County
## 24 2017/18 St Johnstone
## 25 2018/19  Aberdeen
```

## 26	2018/19	Celtic
## 27	2018/19	Dundee
## 28	2018/19	Hamilton
## 29	2018/19	Hearts
## 30	2018/19	Hibernian
## 31	2018/19	Kilmarnock
## 32	2018/19	Livingston
## 33	2018/19	Motherwell
## 34	2018/19	Rangers
## 35	2018/19	St Johnstone
## 36	2018/19	St Mirren
## 37	2019/20	Aberdeen
## 38	2019/20	Celtic
## 39	2019/20	Hamilton
## 40	2019/20	Hearts
## 41	2019/20	Hibernian
## 42	2019/20	Kilmarnock
## 43	2019/20	Livingston
## 44	2019/20	Motherwell
## 45	2019/20	Rangers
## 46	2019/20	Ross County
## 47	2019/20	St Johnstone
## 48	2019/20	St Mirren
## 49	2020/21	Aberdeen
## 50	2020/21	Celtic
## 51	2020/21	Dundee United
## 52	2020/21	Hamilton
## 53	2020/21	Hibernian
## 54	2020/21	Kilmarnock
## 55	2020/21	Livingston
## 56	2020/21	Motherwell
## 57	2020/21	Rangers
## 58	2020/21	Ross County
## 59	2020/21	St Johnstone
## 60	2020/21	St Mirren
## 61	2021/22	Aberdeen
## 62	2021/22	Celtic
## 63	2021/22	Dundee
## 64	2021/22	Dundee United
## 65	2021/22	Hearts
## 66	2021/22	Hibernian
## 67	2021/22	Livingston
## 68	2021/22	Motherwell
## 69	2021/22	Rangers
## 70	2021/22	Ross County
## 71	2021/22	St Johnstone
## 72	2021/22	St Mirren
## 73	2022/23	Aberdeen
## 74	2022/23	Celtic
## 75	2022/23	Dundee United
## 76	2022/23	Hearts
## 77	2022/23	Hibernian
## 78	2022/23	Kilmarnock
## 79	2022/23	Livingston

```
## 80 2022/23      Motherwell
## 81 2022/23      Rangers
## 82 2022/23      Ross County
## 83 2022/23      St Johnstone
## 84 2022/23      St Mirren
## 85 2023/24      Aberdeen
## 86 2023/24      Celtic
## 87 2023/24      Dundee
## 88 2023/24      Hearts
## 89 2023/24      Hibernian
## 90 2023/24      Kilmarnock
## 91 2023/24      Livingston
## 92 2023/24      Motherwell
## 93 2023/24      Rangers
## 94 2023/24      Ross County
## 95 2023/24      St Johnstone
## 96 2023/24      St Mirren
## 97 2024/25      Aberdeen
## 98 2024/25      Celtic
## 99 2024/25      Dundee
## 100 2024/25     Dundee United
## 101 2024/25     Hearts
## 102 2024/25     Hibernian
## 103 2024/25     Kilmarnock
## 104 2024/25     Motherwell
## 105 2024/25     Rangers
## 106 2024/25     Ross County
## 107 2024/25     St Johnstone
## 108 2024/25     St Mirren
## 109 2025/26     Aberdeen
## 110 2025/26     Celtic
## 111 2025/26     Dundee
## 112 2025/26     Dundee United
## 113 2025/26     Falkirk
## 114 2025/26     Hearts
## 115 2025/26     Hibernian
## 116 2025/26     Kilmarnock
## 117 2025/26     Livingston
## 118 2025/26     Motherwell
## 119 2025/26     Rangers
## 120 2025/26     St Mirren
```

```
all_team_names <- sort(unique(team_names_table$Team))
```

```
all_team_names
```

```
## [1] "Aberdeen"      "Celtic"         "Dundee"         "Dundee United"
## [5] "Falkirk"       "Hamilton"       "Hearts"         "Hibernian"
## [9] "Inverness CT"  "Kilmarnock"     "Livingston"     "Motherwell"
## [13] "Partick Thistle" "Rangers"        "Ross County"    "St Johnstone"
## [17] "St Mirren"
```

```
teams_per_season <- team_names_table %>%
  group_by(Season) %>%
  summarise(
```

```

    NumberOfTeams = n_distinct(Team),
    Teams = paste(sort(unique(Team)), collapse = ", "),
    .groups = "drop"
  )
}

teams_per_season

## # A tibble: 10 x 3
##   Season   NumberOfTeams Teams
##   <chr>         <int> <chr>
## 1 2016/17             12 Aberdeen, Celtic, Dundee, Hamilton, Hearts, Inverness ~
## 2 2017/18             12 Aberdeen, Celtic, Dundee, Hamilton, Hearts, Hibernian, ~
## 3 2018/19             12 Aberdeen, Celtic, Dundee, Hamilton, Hearts, Hibernian, ~
## 4 2019/20             12 Aberdeen, Celtic, Hamilton, Hearts, Hibernian, Kilmarnoc ~
## 5 2020/21             12 Aberdeen, Celtic, Dundee United, Hamilton, Hibernian, ~
## 6 2021/22             12 Aberdeen, Celtic, Dundee, Dundee United, Hearts, Hiber~
## 7 2022/23             12 Aberdeen, Celtic, Dundee United, Hearts, Hibernian, Ki~
## 8 2023/24             12 Aberdeen, Celtic, Dundee, Hearts, Hibernian, Kilmarnoc~
## 9 2024/25             12 Aberdeen, Celtic, Dundee, Dundee United, Hearts, Hiber~
## 10 2025/26            12 Aberdeen, Celtic, Dundee, Dundee United, Falkirk, Hear~
options(scipen = 999)

```

Visualisation of match outcomes

```

# Combine the Season and Result columns from each individual season dataset.
# Only these two columns are needed for the match outcome distribution plot.
all_seasons_results <- bind_rows(
  spfl_2016_2017 %>% select(Season, Result),
  spfl_2017_2018 %>% select(Season, Result),
  spfl_2018_2019 %>% select(Season, Result),
  spfl_2019_2020 %>% select(Season, Result),
  spfl_2020_2021 %>% select(Season, Result),
  spfl_2021_2022 %>% select(Season, Result),
  spfl_2022_2023 %>% select(Season, Result),
  spfl_2023_2024 %>% select(Season, Result),
  spfl_2024_2025 %>% select(Season, Result),
  spfl_2025_2026 %>% select(Season, Result)
)

# Count the number of home wins, draws and away wins in each season.
match_outcomes_by_season <- all_seasons_results %>%

  # Keep only valid match result categories.
  filter(Result %in% c("Home", "Away", "Draw")) %>%

  # Set the order in which match results should appear in the plot.
  mutate(
    Result = factor(Result, levels = c("Home", "Draw", "Away"))
  ) %>%

  # Group by season and match outcome, then count the number of matches.
  group_by(Season, Result) %>%
  summarise(

```



```

    NumberOfMatches = n(),
    .groups = "drop"
)

match_outcomes_by_season

## # A tibble: 30 x 3
##   Season Result NumberOfMatches
##   <chr>   <fct>           <int>
## 1 2016/17 Home           94
## 2 2016/17 Draw           58
## 3 2016/17 Away           76
## 4 2017/18 Home           93
## 5 2017/18 Draw           56
## 6 2017/18 Away           79
## 7 2018/19 Home          104
## 8 2018/19 Draw           47
## 9 2018/19 Away           77
## 10 2019/20 Home           79
## # i 20 more rows

# Produce a faceted bar chart showing the distribution of match outcomes
# separately for each SPFL Premiership season.
ggplot(match_outcomes_by_season,
  aes(x = Result,
      y = NumberOfMatches,
      fill = Result)) +
  geom_col() +

  # Add the match count above each bar.
  geom_text(
    aes(label = NumberOfMatches),
    vjust = -0.3,
    size = 3
  ) +

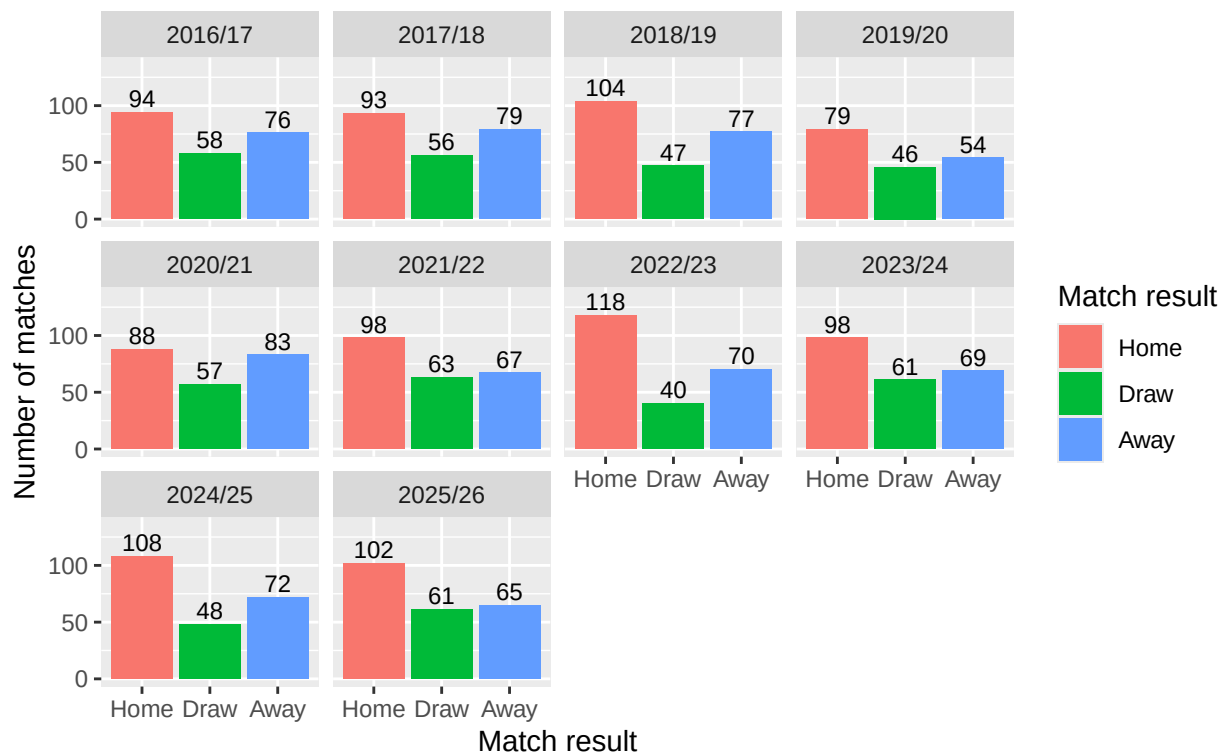
  # Add extra space above the bars so the labels are not cut off.
  scale_y_continuous(
    limits = c(0, max(match_outcomes_by_season$NumberOfMatches) * 1.15)
  ) +

  # Create a separate panel for each season.
  facet_wrap(~ Season) +
  labs(
    title = "Distribution of Match Outcomes by Season",
    subtitle = "SPFL Premiership seasons 2016/17 to 2025/26",
    x = "Match result",
    y = "Number of matches",
    fill = "Match result"
  )

```

Distribution of Match Outcomes by Season

SPFL Premiership seasons 2016/17 to 2025/26



Visualisation of home and away goals

```
library(readxl)
library(dplyr)

# Function to load one season and keep the columns needed
load_goal_difference_data <- function(file, season, sheet = NULL) {

  if (is.null(sheet)) {
    data <- read_excel(file)
  } else {
    data <- read_excel(file, sheet = sheet)
  }

  data %>%
    select(`Home Team`, `Away Team`, `Home Goals`, `Away Goals`, Result) %>%
    mutate(Season = season)
}

# Load each season
spfl_2016_2017 <- load_goal_difference_data("SPFL 2016-2017.xlsx", "2016/17", "SPFL 2016-2017")
spfl_2017_2018 <- load_goal_difference_data("SPFL 2017-2018.xlsx", "2017/18", "SPFL 2017-2018")
spfl_2018_2019 <- load_goal_difference_data("SPFL 2018-2019.xlsx", "2018/19", "SPFL 2018-2019")
spfl_2019_2020 <- load_goal_difference_data("SPFL 2019-2020.xlsx", "2019/20", "SPFL 2019-2020")
spfl_2020_2021 <- load_goal_difference_data("SPFL 2020-2021.xlsx", "2020/21", "SPFL 2020-2021")
```

```

spfl_2021_2022 <- load_goal_difference_data("SPFL 2021-2022.xlsx", "2021/22")
spfl_2022_2023 <- load_goal_difference_data("SPFL 2022-2023.xlsx", "2022/23")
spfl_2023_2024 <- load_goal_difference_data("SPFL 2023-2024.xlsx", "2023/24")
spfl_2024_2025 <- load_goal_difference_data("SPFL 2024-2025.xlsx", "2024/25")
spfl_2025_2026 <- load_goal_difference_data("SPFL 2025-2026.xlsx", "2025/26", "SPFL Dataset")

# Combine all seasons into one dataset
all_goal_difference_data <- bind_rows(
  spfl_2016_2017,
  spfl_2017_2018,
  spfl_2018_2019,
  spfl_2019_2020,
  spfl_2020_2021,
  spfl_2021_2022,
  spfl_2022_2023,
  spfl_2023_2024,
  spfl_2024_2025,
  spfl_2025_2026
)

# Calculate home goal difference
all_goal_difference_data <- all_goal_difference_data %>%
  mutate(
    HomeGoalDifference = `Home Goals` - `Away Goals`
  )

# This code creates a bar plot showing the distribution of goals scored
# by home and away teams across each Scottish Premiership season.

# First, create a new dataset where home goals and away goals are placed
# into one single column called "Goals".
# This makes the data easier to plot using ggplot.
goals_distribution <- data.frame(

  # Repeat the season column twice: once for home goals and once for away goals
  Season = c(all_goal_difference_data$Season,
    all_goal_difference_data$Season),

  # Create a variable showing whether the goals were scored by the home or away team
  Team = c(rep("Home", nrow(all_goal_difference_data)),
    rep("Away", nrow(all_goal_difference_data))),

  # Combine the home goals and away goals into one column
  Goals = c(all_goal_difference_data$`Home Goals`,
    all_goal_difference_data$`Away Goals`)
)

# Count how many times each number of goals was scored by home and away teams
# in each season.
goals_count <- goals_distribution %>%

  # Group the data by season, team type, and number of goals scored
  group_by(Season, Team, Goals) %>%

```

```

# Count the number of matches in each group
summarise(Count = n(), .groups = "drop") %>%

# Group again by season and team so percentages are calculated separately
# for home and away teams within each season
group_by(Season, Team) %>%

# Convert the raw counts into percentages of matches
mutate(Percentage = Count / sum(Count) * 100)

# Create a bar plot showing the percentage distribution of goals scored
# by home and away teams for each season.
ggplot(goals_count, aes(x = factor(Goals, levels = 0:9),
                           y = Percentage,
                           fill = Team)) +

# Create bars using the percentage values already calculated above.
# position = "dodge" places the home and away bars side by side.
geom_bar(stat = "identity", position = "dodge") +

# Split the plot into separate panels for each season.
# ncol = 5 gives two rows of five seasons.
facet_wrap(~ Season, ncol = 5) +

# Keep the x-axis goal values from 0 to 9 visible across all panels,
# even if some goal totals do not occur in a particular season.
scale_x_discrete(drop = FALSE) +

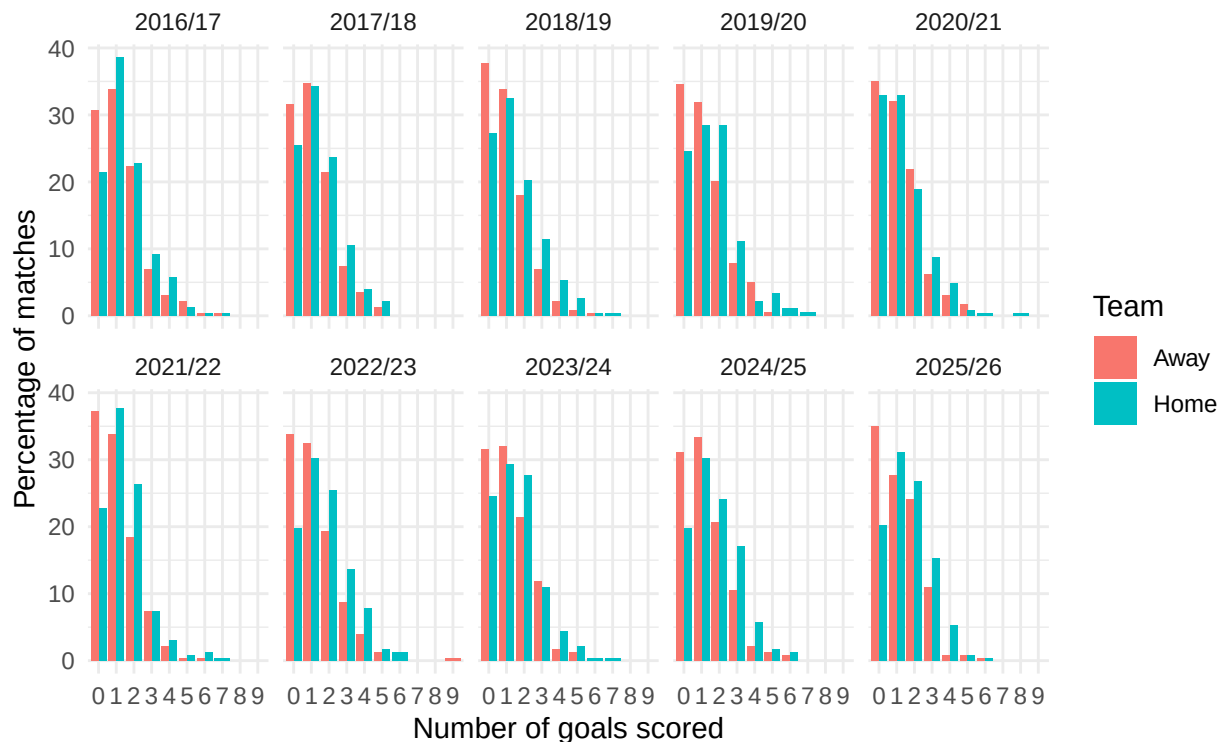
# Add the title, subtitle, axis labels, and legend title.
labs(
  title = "Distribution of Goals Scored by Home and Away Teams",
  subtitle = "SPFL Premiership seasons 2016/17 to 2025/26",
  x = "Number of goals scored",
  y = "Percentage of matches",
  fill = "Team"
) +

# Use a simple, clean theme for the plot.
theme_minimal()

```

Distribution of Goals Scored by Home and Away Teams

SPFL Premiership seasons 2016/17 to 2025/26



```
# Get season names
seasons <- unique(all_goal_difference_data$Season)

# Create a different colour for each season
cols <- c("red", "orange", "gold", "darkgreen", "turquoise4",
          "deepskyblue", "blue", "purple", "black", "magenta")

# Leave some extra room on the right for the legend
par(pty = "s", mar = c(5, 5, 4, 8))

# Create empty plot
plot(0, 0,
     type = "n",
     asp = 1,
     xlim = c(0, 400),
     ylim = c(0, 400),
     xlab = "Cumulative home goals",
     ylab = "Cumulative away goals",
     main = "Cumulative Home and Away Goals")

# Equality line
abline(0, 1, lty = 2)

# Plot each season
for (i in seq_along(seasons)) {

  season_data <- all_goal_difference_data[
```

```

all_goal_difference_data$Season == seasons[i], ]

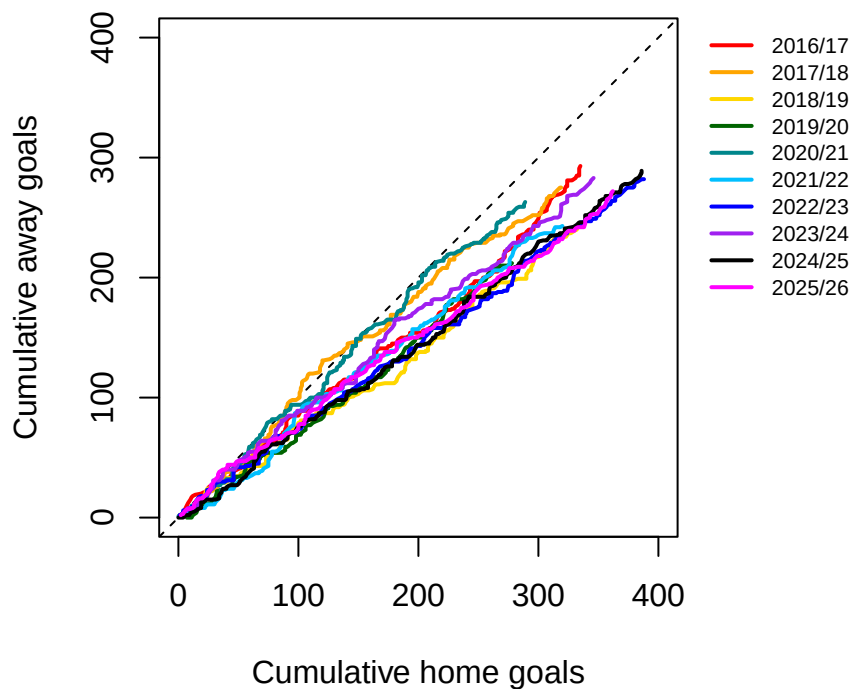
home <- season_data$`Home Goals`
away <- season_data$`Away Goals`

lines(cumsum(home),
      cumsum(away),
      col = cols[i],
      lwd = 2)
}

# Put legend outside the graph
legend("topright",
      inset = c(-0.35, 0),
      xpd = TRUE,
      legend = seasons,
      col = cols,
      lty = 1,
      lwd = 2,
      cex = 0.7,
      bty = "n")

```

Cumulative Home and Away Goals



Attendance and match outcome

Visualisation of attendance by match result

```

# Produce a boxplot showing how attendance varies across match outcomes.
# This compares the distribution of attendances for home wins, draws and away wins.

```

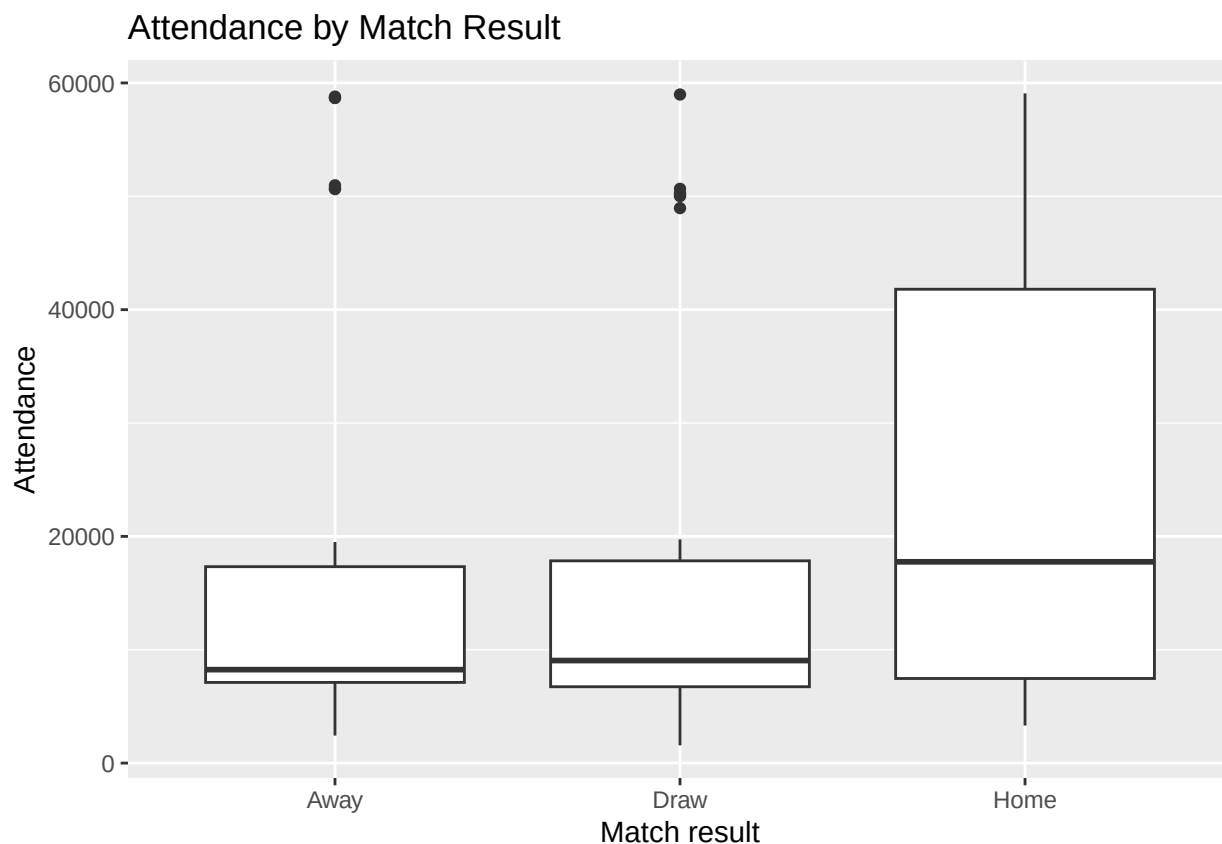
```

spfl <- read_excel("SPFL 2025-2026.xlsx", sheet = "SPFL Dataset")
ggplot(spfl, aes(x = Result, y = Attendance)) +

  # Add boxplots to show the median, interquartile range and possible outliers
  # in attendance for each match result category.
  geom_boxplot() +

  # Add plot labels.
  labs(
    title = "Attendance by Match Result",
    x = "Match result",
    y = "Attendance"
  )

```



Logistic regression: do percent capacity and travel distance affect the probability of a home win?

A logistic regression model was fitted because HomeWin is a binary response variable. The generalised linear model keeps predicted probabilities between 0 and 1.

```

# Create a binary outcome variable for whether the home team won.
# Home wins are coded as 1, while away wins and draws are coded as 0.
# Any missing or unexpected result values are coded as NA.
spfl$HomeWin <- ifelse(spfl$Result == "Home", 1,
  ifelse(spfl$Result %in% c("Away", "Draw"), 0, NA))

```

```

# Fit a logistic regression model to examine whether percentage capacity
# and away-team travel distance are associated with the probability
# of a home win.
logit_model <- glm(
  HomeWin ~ `Percent Capacity` + `Travel Distance for Away Team`,
  data = spfl,
  family = "binomial"
)

# Display the model output, including coefficient estimates,
# standard errors, z-values and p-values.
summary(logit_model)

##
## Call:
## glm(formula = HomeWin ~ `Percent Capacity` + `Travel Distance for Away Team`,
##      family = "binomial", data = spfl)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -1.521117   0.592337  -2.568   0.0102 *
## `Percent Capacity`    1.247807   0.636133   1.962   0.0498 *
## `Travel Distance for Away Team`  0.005562   0.003317   1.677   0.0936 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 313.54  on 227  degrees of freedom
## Residual deviance: 307.78  on 225  degrees of freedom
## AIC: 313.78
##
## Number of Fisher Scoring iterations: 4
# Show one-sided p-values
pnorm(
  summary(logit_model)$coefficients[, "z value"],
  lower.tail = FALSE
)

```

```

##              (Intercept)              `Percent Capacity`
##              0.99488553              0.02490746
## `Travel Distance for Away Team`
##              0.04681547

```

One-sided hypothesis tests were used because the analysis tested the directional hypotheses that greater percentage capacity and greater away-team travel distance would increase the probability of a home win.

Visualisation of percent capacity and home wins

```

ggplot(spfl, aes(x = `Percent Capacity`, y = HomeWin)) +
  geom_jitter(height = 0.05) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"), se = FALSE) +
  labs(
    title = "Percent Capacity and Home Wins",

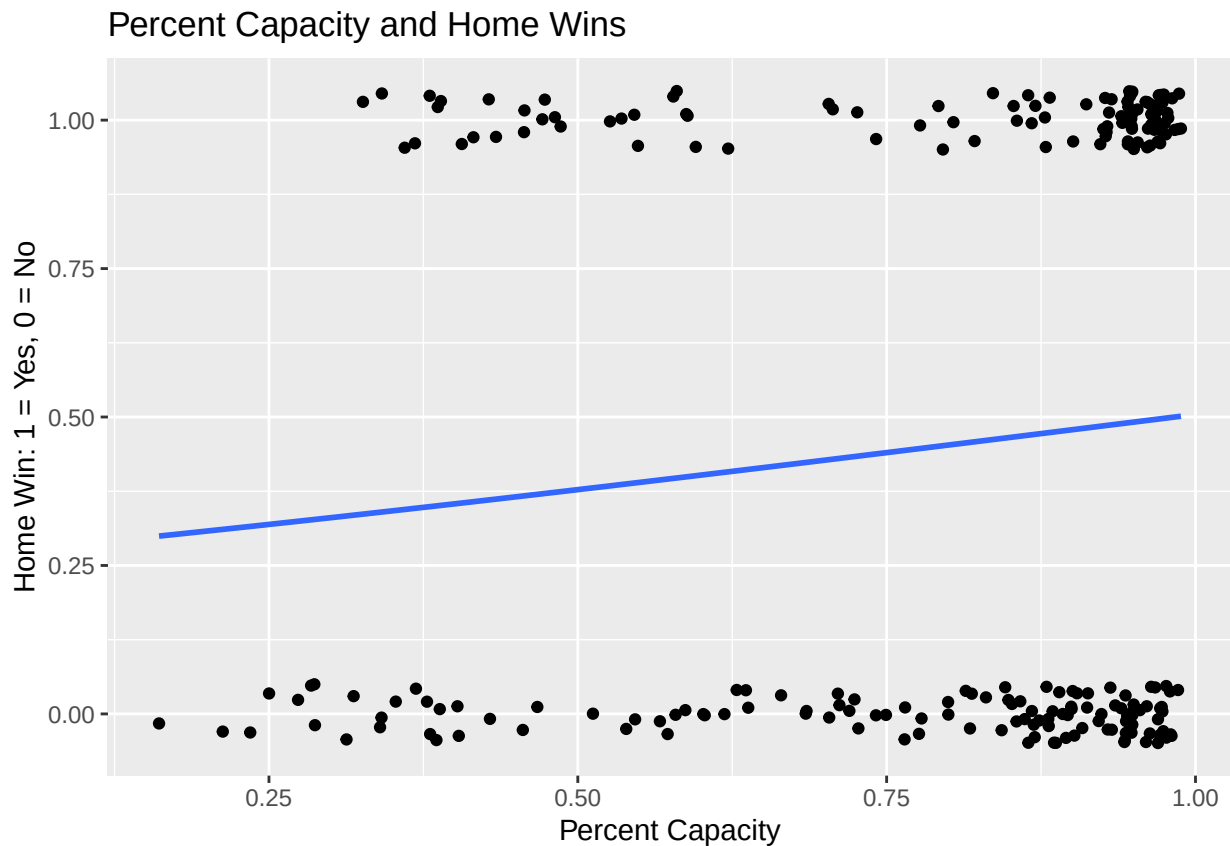
```



```

x = "Percent Capacity",
y = "Home Win: 1 = Yes, 0 = No"
)

```



Visualisation of travel distance and home wins

```

# Produce a plot showing the relationship between away-team travel distance
# and whether the home team won the match.
ggplot(spf1, aes(x = `Travel Distance for Away Team`, y = HomeWin)) +

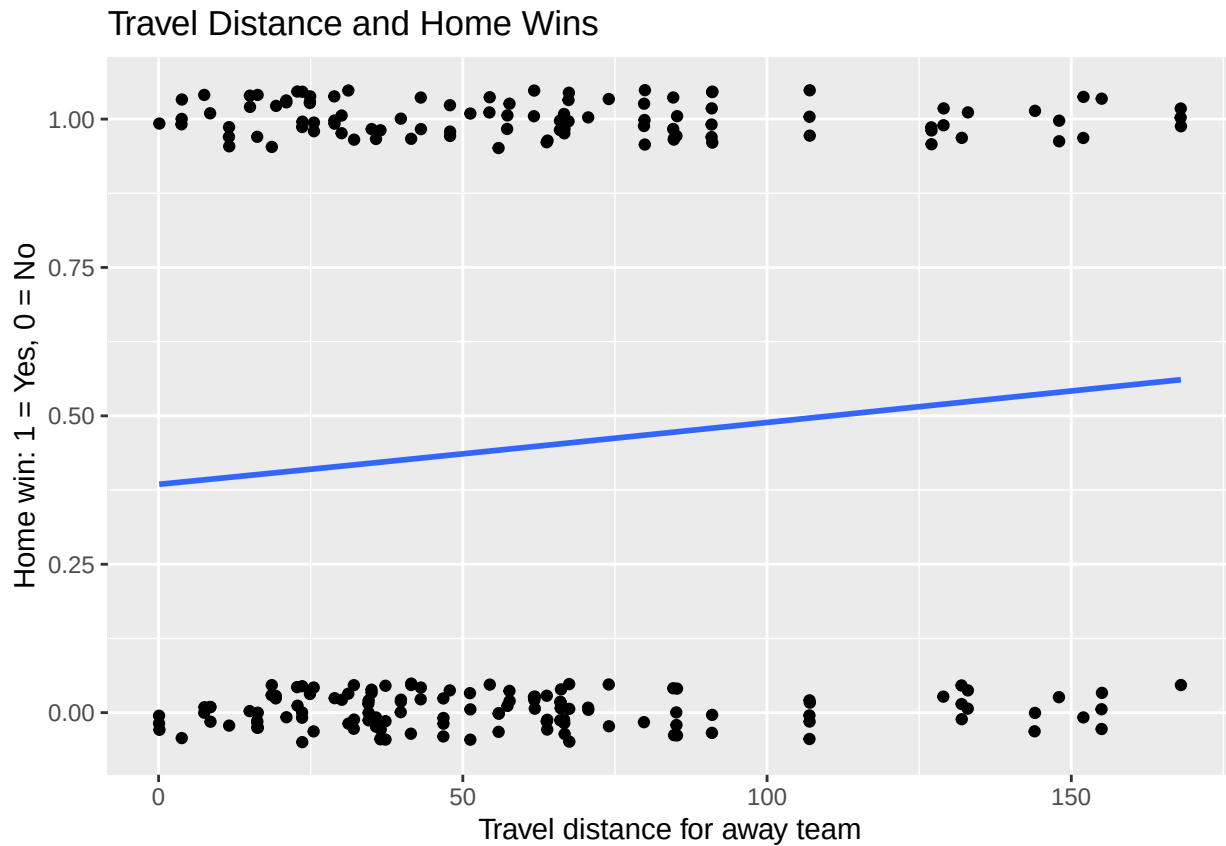
  # Add jittered points so that overlapping 0 and 1 values are easier to see.
  # HomeWin is binary, where 1 indicates a home win and 0 indicates no home win.
  geom_jitter(height = 0.05) +

  # Add a logistic regression trend line because the response variable,
  # HomeWin, is binary.
  geom_smooth(
    method = "glm",
    method.args = list(family = "binomial"),
    se = FALSE
  ) +

  # Add plot labels.
  labs(
    title = "Travel Distance and Home Wins",
    x = "Travel distance for away team",

```

```
y = "Home win: 1 = Yes, 0 = No"
)
```



Disciplinary analysis: did away teams receive more cards?

Distribution of yellow cards per match

```
home_yellows <- table(spfl$`Home Yellow cards`)
away_yellows <- table(spfl$`Away Yellow cards`)
```

```
home_yellows
```

```
##
##  0  1  2  3  4  5  6  7
## 38 60 69 39 17  2  2  1
```

```
away_yellows
```

```
##
##  0  1  2  3  4  5  6
## 23 55 68 45 23 10  4
```

```
home_yellows_covid <- table(spfl_2020_2021$`Home Yellow cards`)
away_yellows_covid <- table(spfl_2020_2021$`Away Yellow cards`)
```

```
yellow_values <- 0:max(
  spfl$`Home Yellow cards`,
```

```

    spfl$`Away Yellow cards`,
    na.rm = TRUE
)

yellow_card_table <- data.frame(
  YellowCards = yellow_values,
  Home = as.numeric(table(factor(spfl$`Home Yellow cards`,
                                levels = yellow_values))),
  Away = as.numeric(table(factor(spfl$`Away Yellow cards`,
                                levels = yellow_values)))
)

yellow_card_table

##   YellowCards Home Away
## 1           0   38   23
## 2           1   60   55
## 3           2   69   68
## 4           3   39   45
## 5           4   17   23
## 6           5    2   10
## 7           6    2    4
## 8           7    1    0

# Reload full 2020/21 dataset because yellow-card columns are needed
spfl_2020_full <- read_excel("SPFL 2020-2021.xlsx", sheet = "SPFL 2020-2021")

# Check exact column names if needed
names(spfl_2020_full)

## [1] "Date"           "Home Team"      "Away Team"
## [4] "Home Goals"     "Away Goals"     "Result"
## [7] "Goal Difference" "Home Yellow cards" "Home Red cards"
## [10] "Away Yellow cards" "Away Red cards"

# Create yellow card values
yellow_values_covid <- 0:max(
  c(
    spfl_2020_full$`Home Yellow cards`,
    spfl_2020_full$`Away Yellow cards`
  ),
  na.rm = TRUE
)

# Combined yellow-card table
yellow_card_table_covid <- data.frame(
  YellowCards = yellow_values_covid,
  Home = as.numeric(table(factor(
    spfl_2020_full$`Home Yellow cards`,
    levels = yellow_values_covid
  ))),
  Away = as.numeric(table(factor(
    spfl_2020_full$`Away Yellow cards`,
    levels = yellow_values_covid
  )))
)

```

```
)
```

```
yellow_card_table_covid
```

```
##   YellowCards Home Away
## 1           0   44   39
## 2           1   83   86
## 3           2   51   55
## 4           3   32   31
## 5           4   11   11
## 6           5    4    5
## 7           6    3    1
```

Visualisation of yellow cards

```
# Reload the full 2020/21 dataset because the yellow-card columns
# are required for this analysis.
spfl_2020_full <- read_excel("SPFL 2020-2021.xlsx", sheet = "SPFL 2020-2021")

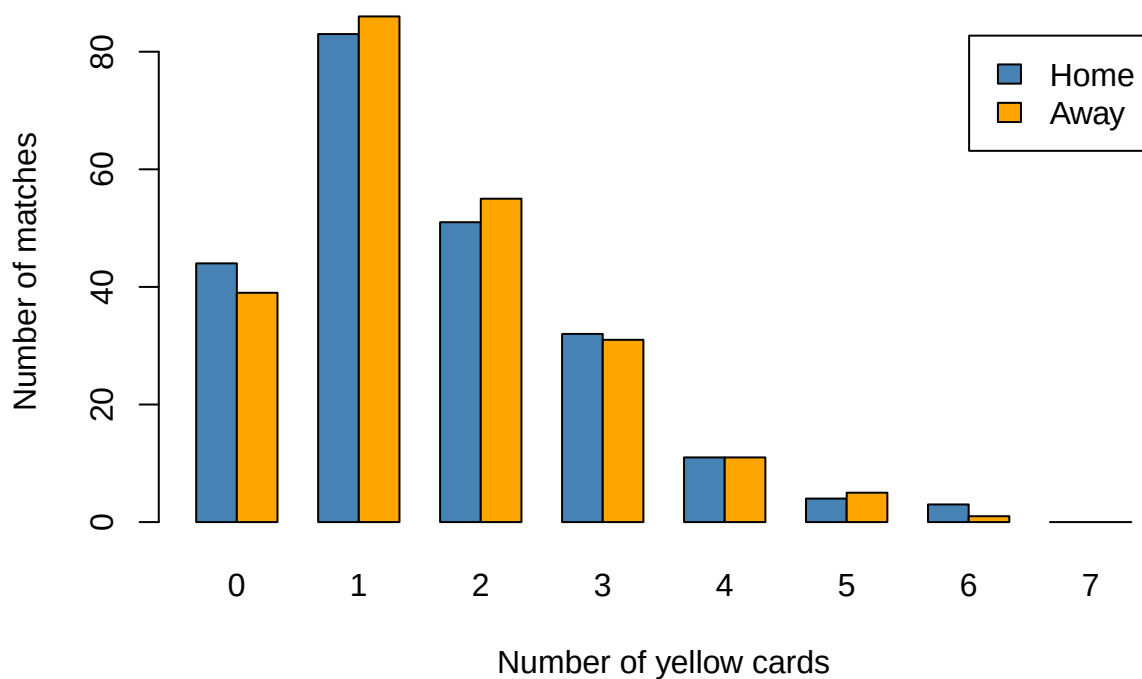
# Define the possible yellow-card values to be displayed on the plot.
# This ensures that all values from 0 to 7 appear, even if some have zero counts.
# Most yellow cards in a single game was 7
yellow_values_covid <- 0:7

# Count how many matches had each number of home-team yellow cards.
home_yellows_covid <- table(
  factor(spfl_2020_full$`Home Yellow cards`, levels = yellow_values_covid)
)

# Count how many matches had each number of away-team yellow cards.
away_yellows_covid <- table(
  factor(spfl_2020_full$`Away Yellow cards`, levels = yellow_values_covid)
)

# Produce a grouped bar chart comparing the distribution of yellow cards
# for home and away teams during the 2020/21 season.
barplot(
  rbind(home_yellows_covid, away_yellows_covid),
  beside = TRUE,
  legend.text = c("Home", "Away"),
  col = c("steelblue", "orange"),
  main = "Distribution of Yellow Cards for Home and Away Teams: 2020/21",
  xlab = "Number of yellow cards",
  ylab = "Number of matches"
)
```

Distribution of Yellow Cards for Home and Away Teams: 2020/21



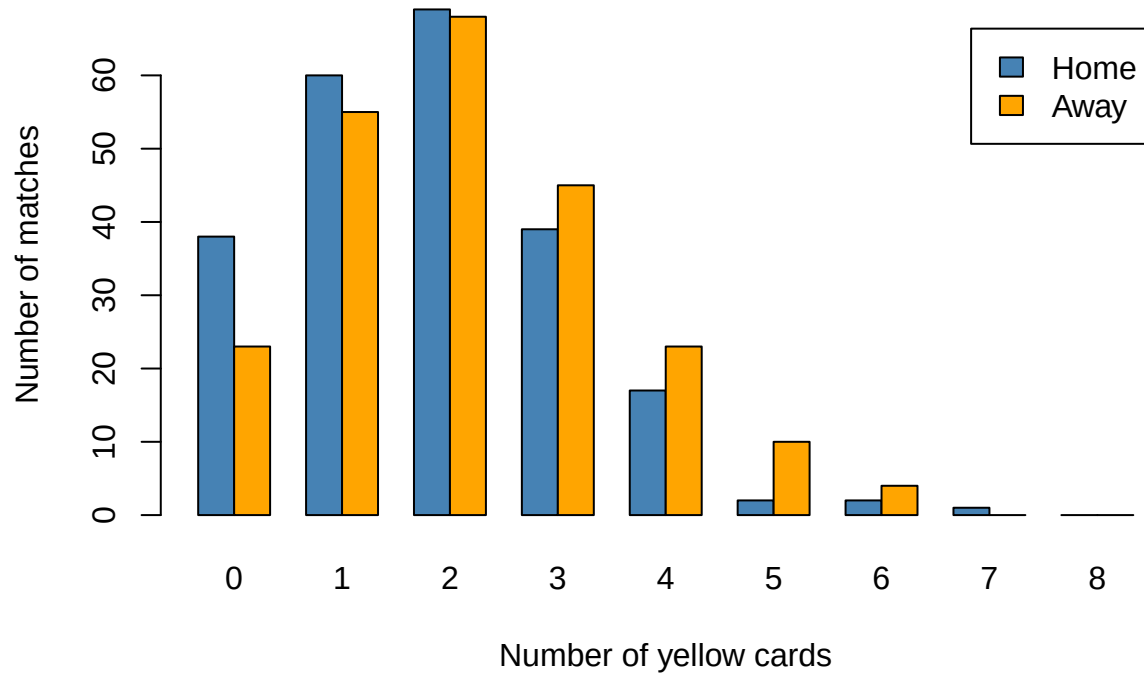
```
# Define the possible yellow-card values to be displayed on the plot.
# This keeps the x-axis consistent, including values with zero matches.
yellow_values <- 0:8

# Count how many 2025/26 matches had each number of home-team yellow cards.
home_yellows <- table(
  factor(spl$`Home Yellow cards`, levels = yellow_values)
)

# Count how many 2025/26 matches had each number of away-team yellow cards.
away_yellows <- table(
  factor(spl$`Away Yellow cards`, levels = yellow_values)
)

# Produce a grouped bar chart comparing the distribution of yellow cards
# for home and away teams during the 2025/26 season.
barplot(
  rbind(home_yellows, away_yellows),
  beside = TRUE,
  legend.text = c("Home", "Away"),
  col = c("steelblue", "orange"),
  main = "Distribution of Yellow Cards for Home and Away Teams: 2025/26",
  xlab = "Number of yellow cards",
  ylab = "Number of matches"
)
```

Distribution of Yellow Cards for Home and Away Teams: 2025/26



```
yellow_values <- 0:max(
  spfl$`Home Yellow cards`,
  spfl$`Away Yellow cards`,
  na.rm = TRUE
)

yellow_card_table <- data.frame(
  YellowCards = yellow_values,
  Home = as.numeric(table(factor(spfl$`Home Yellow cards`,
                                levels = yellow_values))),
  Away = as.numeric(table(factor(spfl$`Away Yellow cards`,
                                levels = yellow_values)))
)

yellow_card_table
```

```
##   YellowCards Home Away
## 1          0   38   23
## 2          1   60   55
## 3          2   69   68
## 4          3   39   45
## 5          4   17   23
## 6          5    2   10
## 7          6    2    4
## 8          7    1    0
```

Red cards

```
home_recs <- table(spfl$`Home Red cards`)
away_recs <- table(spfl$`Away Red cards`)
```

```
home_recs
```

```
##
##    0    1    2
## 209   17    2
```

```
away_recs
```

```
##
##    0    1
## 212   16
```

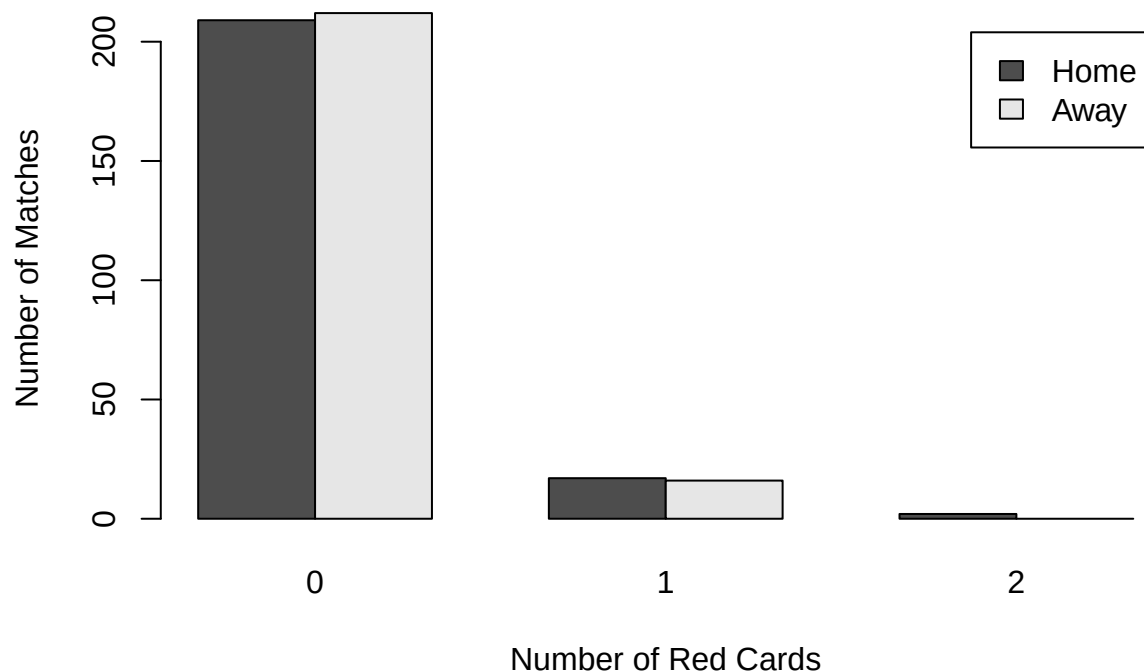
Visualisation of red cards

```
red_values <- 0:2
```

```
home_recs <- table(factor(spfl$`Home Red cards`, levels = red_values))
away_recs <- table(factor(spfl$`Away Red cards`, levels = red_values))
```

```
barplot(
  rbind(home_recs, away_recs),
  beside = TRUE,
  legend.text = c("Home", "Away"),
  main = "Distribution of Red Cards for Home and Away Teams",
  xlab = "Number of Red Cards",
  ylab = "Number of Matches"
)
```

Distribution of Red Cards for Home and Away Teams



Key Match incident(KMI) data for 2024-25, 2025-26

```
# Load packages needed for data manipulation and plotting.
library(dplyr)
library(ggplot2)

# Create a dataset of incorrect Key Match Incident decisions for 2024/25.
# DecisionsFor records incorrect decisions that benefited each team.
# DecisionsAgainst records incorrect decisions that went against each team.
kmi_2024_2025 <- data.frame(
  Season = "2024/25",
  Team = c("Celtic", "Rangers", "Hibernian", "Dundee United",
           "Aberdeen", "St Mirren", "Hearts", "Motherwell",
           "Kilmarnock", "Dundee", "Ross County", "St Johnstone"),
  DecisionsFor = c(3, 2, 4, 3, 1, 4, 1, 2, 1, 1, 3, 3),
  DecisionsAgainst = c(2, 5, 0, 2, 1, 5, 4, 3, 4, 0, 2, 0)
)

# Create the equivalent KMI dataset for 2025/26.
kmi_2025_2026 <- data.frame(
  Season = "2025/26",
  Team = c("Celtic", "Hearts", "Rangers", "Motherwell",
           "Hibernian", "Falkirk", "Dundee United", "Dundee",
           "Aberdeen", "Kilmarnock", "St Mirren", "Livingston"),
  DecisionsFor = c(2, 1, 1, 2, 1, 3, 0, 0, 0, 2, 2, 3),
  DecisionsAgainst = c(1, 1, 3, 4, 1, 0, 1, 1, 1, 0, 3, 1)
)

# Combine both seasons into one dataset so that the KMI data
# can be analysed across the two seasons together.
kmi_all <- bind_rows(kmi_2024_2025, kmi_2025_2026)

# Summarise the combined KMI data by team.
# This gives the total number of incorrect decisions for and against
# each team across the 2024/25 and 2025/26 seasons.
kmi_all_teams <- kmi_all %>%
  group_by(Team) %>%
  summarise(
    TotalFor = sum(DecisionsFor, na.rm = TRUE),
    TotalAgainst = sum(DecisionsAgainst, na.rm = TRUE),
    NetDifferential = TotalFor - TotalAgainst,
    .groups = "drop"
  ) %>%

  # Order teams from the highest to lowest net differential.
  arrange(desc(NetDifferential))

# Display the team-level summary table.
kmi_all_teams
```

```
## # A tibble: 14 x 4
##   Team          TotalFor TotalAgainst NetDifferential
##   <chr>          <dbl>         <dbl>         <dbl>
## 1 Hibernian          5             1             4
```



```
## 2 Falkirk          3          0          3
## 3 St Johnstone     3          0          3
## 4 Celtic           5          3          2
## 5 Livingston       3          1          2
## 6 Ross County      3          2          1
## 7 Dundee           1          1          0
## 8 Dundee United    3          3          0
## 9 Aberdeen         1          2         -1
## 10 Kilmarnock       3          4         -1
## 11 St Mirren        6          8         -2
## 12 Hearts           2          5         -3
## 13 Motherwell       4          7         -3
## 14 Rangers          3          8         -5
```

```
# Carry out a chi-squared goodness-of-fit test on the distribution
# of incorrect decisions against teams.
chisq.test(kmi_all_teams$TotalAgainst)
```

```
##
## Chi-squared test for given probabilities
##
## data: kmi_all_teams$TotalAgainst
## X-squared = 31.844, df = 13, p-value = 0.002533
```

```
# Produce a bar chart showing the net differential of incorrect KMI decisions
# for each team across the combined 2024/25 and 2025/26 data.
```

```
ggplot(kmi_all_teams,
       aes(x = reorder(Team, NetDifferential),
           y = NetDifferential)) +
```

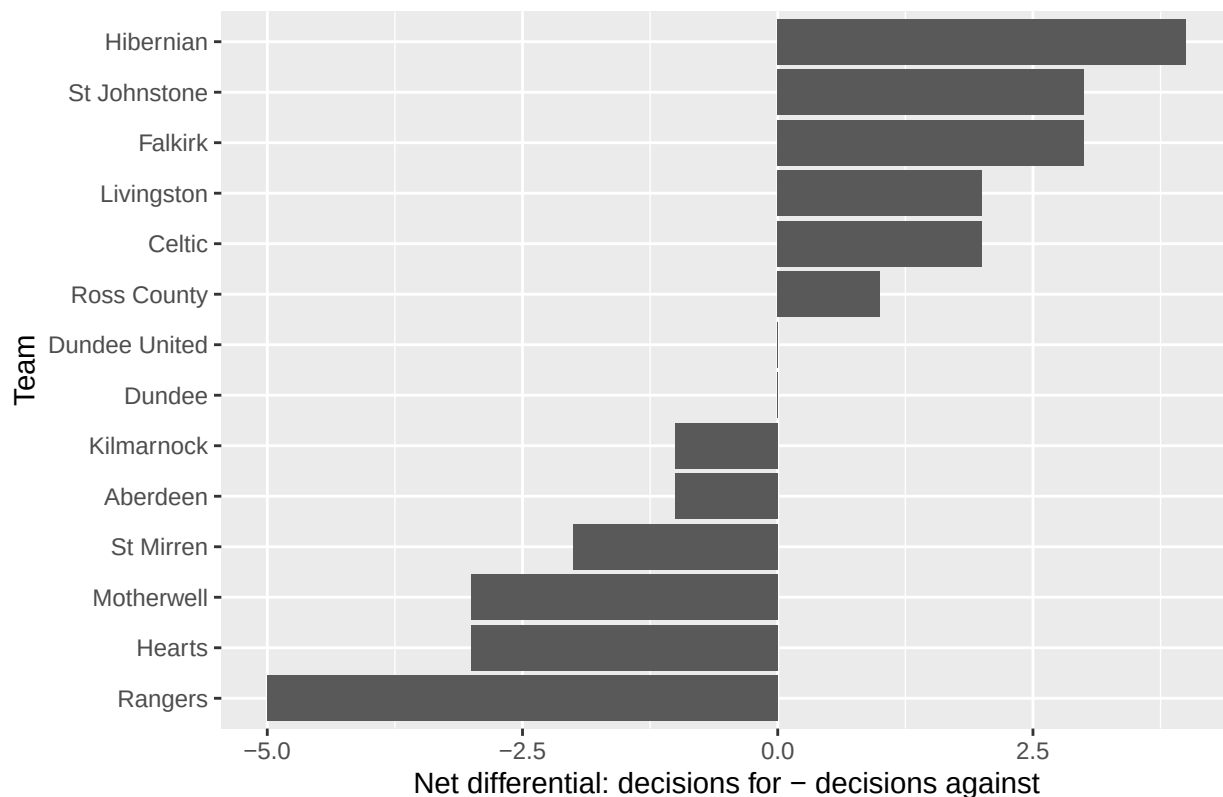
```
# Add bars representing each team's net differential.
geom_col() +
```

```
# Flip the axes so that team names are easier to read.
coord_flip() +
```

```
# Add plot labels.
```

```
labs(
  title = "Net Differential of Incorrect KMI Decisions by Team",
  x = "Team",
  y = "Net differential: decisions for - decisions against"
)
```

Net Differential of Incorrect KMI Decisions by Team



```
# Create team-level totals from the combined KMI dataset.
# This calculates the total number of incorrect decisions for and against
# each team across the two seasons.
kmi_all_teams <- kmi_all %>%
  group_by(Team) %>%
  summarise(
    TotalFor = sum(DecisionsFor, na.rm = TRUE),
    TotalAgainst = sum(DecisionsAgainst, na.rm = TRUE),
    TotalDecisions = TotalFor + TotalAgainst,
    NetDifferential = TotalFor - TotalAgainst,
    .groups = "drop"
  )

# Carry out an exact binomial test for each team.
# The test checks whether the proportion of incorrect decisions against
# a team differs significantly from 50% of all incorrect decisions involving
# that team.
kmi_binomial_tests <- kmi_all_teams %>%
  rowwise() %>%
  mutate(

    # TotalAgainst is treated as the number of "successes",
    # while TotalDecisions is the total number of incorrect decisions
    # for and against that team.
    PValue = binom.test(
      x = TotalAgainst,
      n = TotalDecisions,
```

```

    p = 0.5,
    alternative = "two.sided"
  )$p.value
) %>%
ungroup() %>%

# Adjust the p-values using the Holm method to account for
# multiple team-level tests.
mutate(
  AdjustedPValue = p.adjust(PValue, method = "holm")
) %>%

# Order the results from smallest to largest unadjusted p-value.
arrange(PValue)

# Display the binomial test results.
kmi_binomial_tests

## # A tibble: 14 x 7
##   Team          TotalFor TotalAgainst TotalDecisions NetDifferential PValue
##   <chr>          <dbl>         <dbl>         <dbl>          <dbl> <dbl>
## 1 Hibernian      5             1             6             4 0.219
## 2 Rangers        3             8            11            -5 0.227
## 3 Falkirk         3             0             3             3 0.25
## 4 St Johnstone   3             0             3             3 0.25
## 5 Hearts         2             5             7            -3 0.453
## 6 Motherwell     4             7            11            -3 0.549
## 7 Livingston     3             1             4             2 0.625
## 8 Celtic         5             3             8             2 0.727
## 9 St Mirren      6             8            14            -2 0.791
## 10 Aberdeen      1             2             3            -1 1
## 11 Dundee         1             1             2             0 1
## 12 Dundee United  3             3             6             0 1
## 13 Kilmarnock     3             4             7            -1 1
## 14 Ross County   3             2             5             1 1
## # i 1 more variable: AdjustedPValue <dbl>

```